Ejercicio #6

1.1. Descripción general

El ejercicio plantea desarrollar un sistema que permita administrar las integraciones de una plataforma de comercio electrónico con diferentes servicios externos, como pasarelas de pago, proveedores de logística y certificadores de Factura Electrónica en Línea (FEL).

El principal desafío consiste en gestionar estos servicios desde un sistema centralizado, considerando que cada proveedor ofrece funcionalidades diferentes. Algunas integraciones permiten realizar operaciones financieras, como cobros y reintegros; otras se encargan de consultar estados y cotizar envíos; mientras que algunas permiten generar documentos electrónicos y registrar operaciones.

Para resolver esta situación, se propone implementar una estructura basada en programación orientada a objetos, utilizando herencia para representar las características compartidas entre las integraciones e interfaces para establecer los comportamientos específicos que puede ofrecer cada servicio.

El sistema deberá permitir registrar al menos diez integraciones, consultar su información, verificar su estado de conexión y ordenar el catálogo según su latencia promedio. Además, deberá contar con un menú de consola que facilite la interacción con el administrador.

Se busca que la estructura sea flexible y permita incorporar nuevos proveedores en el futuro sin modificar la lógica principal del sistema.

1.2. Requisitos funcionales

A partir del problema planteado, se identifican los siguientes requisitos funcionales:

Código	Requisito funcional
RF01	El sistema deberá registrar al menos diez integraciones con diferentes servicios externos.
RF02	Cada integración deberá contar con un identificador único para facilitar su búsqueda.
RF03	El sistema deberá almacenar información general de cada proveedor, incluyendo su nombre, estado de conexión y latencia promedio.

RF04	El sistema deberá permitir listar todas las integraciones registradas, mostrando su información y estado de conexión.
RF05	El sistema deberá permitir buscar una integración mediante su identificador y consultar sus parámetros de configuración.
RF06	El sistema deberá ordenar las integraciones según su latencia promedio, de menor a mayor.
RF07	El sistema deberá permitir realizar operaciones de cobro y reintegro únicamente mediante integraciones que ofrezcan funcionalidades transaccionales.
RF08	El sistema deberá permitir consultar el estado de las integraciones que implementen dicha funcionalidad.
RF09	El sistema deberá permitir realizar cotizaciones mediante los proveedores que ofrezcan este servicio.
RF10	El sistema deberá permitir generar documentos electrónicos mediante las integraciones que cuenten con esta capacidad.
RF11	El sistema deberá registrar las operaciones realizadas mediante las integraciones que permitan auditoría.
RF12	El sistema deberá contar con un menú de consola que permita al administrador acceder a las funcionalidades disponibles.

1.3. Identificación preliminar de clases

Para desarrollar el sistema se identifican las siguientes clases e interfaces, considerando las responsabilidades y funcionalidades necesarias para resolver el problema.

Clases principales:

- **IntegracionAPI:** clase abstracta que representa la estructura general de las integraciones y contiene los atributos y comportamientos compartidos.

- **PasarelaPago:** representa los servicios externos encargados de procesar operaciones financieras.
- **ProveedorLogistico:** representa los servicios encargados de gestionar envíos, consultar estados y realizar cotizaciones.
- **CertificadorFEL:** representa los servicios encargados de generar documentos electrónicos y registrar operaciones.
- **IntegracionController:** administra el conjunto de integraciones y coordina las operaciones disponibles.
- **MenuConsola:** permite al administrador interactuar con el sistema mediante un menú de opciones.
- **Main:** contiene el punto de entrada del programa y se encarga de iniciar los componentes principales.

Interfaces:

- **Transaccional:** establece las operaciones financieras que puede realizar una integración.
- **Documentable:** define la capacidad de generar documentos electrónicos.
- **Auditable:** establece la funcionalidad de registrar las operaciones realizadas.
- **Consultable:** define la capacidad de consultar el estado de una integración.
- **Cotizable:** establece la capacidad de calcular el costo de un envío.

1.4. Propósito de cada clase

Clase	Propósito
IntegracionAPI	Representar las características generales de una integración externa, centralizando la información compartida por los diferentes proveedores.
PasarelaPago	Gestionar las operaciones financieras de los servicios de pago, incluyendo cobros, reintegros y consultas de estado.

ProveedorLogistico	Administrar las operaciones relacionadas con los servicios de envío, permitiendo consultar estados, calcular cotizaciones y generar guías de envío.
CertificadorFEL	Representar los servicios encargados de generar documentos electrónicos, consultar estados de certificación y registrar operaciones.
IntegracionController	Administrar la lista polimórfica de integraciones y coordinar las operaciones de búsqueda, consulta, ordenamiento y procesamiento de solicitudes.
MenuConsola	Presentar las opciones disponibles y permitir que el administrador interactúe con las funcionalidades del sistema.
Main	Inicializar el sistema, crear los componentes necesarios y ejecutar el menú principal.

Propósito de las interfaces:

Interfaz	**Propósito**
Transaccional	Establecer un contrato para las integraciones que permiten procesar cobros y reintegros.
Documentable	Definir el comportamiento necesario para generar documentos electrónicos.
Auditable	Permitir que las integraciones registren las operaciones y eventos realizados.
Consultable	Establecer un comportamiento común para consultar el estado de los servicios.
Cotizable	Definir la funcionalidad necesaria para calcular el costo de los envíos.

1.5. Identificación preliminar de atributos

A continuación, se identifican los atributos necesarios para almacenar la información de cada clase.

Clase: IntegracionAPI

Atributo	Tipo de dato	Propósito
id	String	Identificar de manera única cada integración registrada.
nombreProveedor	String	Almacenar el nombre del proveedor del servicio externo.
estadoConexion	String	Indicar el estado actual de la conexión con la API.
latenciaPromedio	double	Almacenar el tiempo promedio de respuesta de la integración en milisegundos.
parametrosConfiguracion	Map	Almacenar los parámetros necesarios para configurar cada integración.

Clase: PasarelaPago

No requiere atributos adicionales en la propuesta inicial, ya que utiliza la información general heredada de IntegracionAPI.

Clase: ProveedorLogistico

No requiere atributos adicionales en la propuesta inicial, ya que utiliza la información general heredada de IntegracionAPI.

Clase: CertificadorFEL

No requiere atributos adicionales en la propuesta inicial, ya que utiliza la información general heredada de IntegracionAPI.

Clase: IntegracionController

Atributo	Tipo de dato	Propósito
integraciones	ArrayList<IntegracionAPI>	Almacenar todas las integraciones registradas mediante una única lista polimórfica.

Clase: MenuConsola

Atributo	Tipo de dato	Propósito
controlador	IntegracionController	Mantener una referencia al controlador para acceder a las funcionalidades del sistema.

Las interfaces no requieren atributos propios, debido a que su función principal es establecer los métodos que deben implementar las clases correspondientes.

1.6. Identificación preliminar de métodos

Se identifican los métodos necesarios para cumplir con las responsabilidades de cada clase y permitir la comunicación entre los componentes del sistema.

Clase: IntegracionAPI

Método	Descripción
IntegracionAPI(id, nombreProveedor)	Constructor que permite inicializar una integración con su identificador y nombre.
IntegracionAPI(id, nombreProveedor, latencia, estado)	Constructor sobrecargado que permite inicializar una integración con información adicional.
getId()	Obtener el identificador de la integración.
getNombreProveedor()	Obtener el nombre del proveedor.
getEstadoConexion()	Consultar el estado de conexión.

getLatenciaPromedio()	Obtener la latencia promedio de la integración.
getParametrosConfiguracion()	Consultar los parámetros de configuración.
setEstadoConexion(estado)	Modificar el estado de conexión.
compareTo(otra)	Comparar dos integraciones según su latencia promedio para facilitar el ordenamiento.
toString()	Devolver una representación textual de la información de la integración.

Clase: PasarelaPago

Método	**Descripción**
cobrar(monto)	Procesar una operación de cobro por el monto indicado.
reintegrar(monto)	Procesar la devolución de un monto previamente cobrado.
consultarEstado()	Consultar el estado de las operaciones o del servicio de pago.
registrarAuditoria(evento)	Registrar los eventos relacionados con las operaciones realizadas.

Clase: ProveedorLogistico

Método	**Descripción**
consultarEstado()	Consultar el estado de un envío o del servicio logístico.
cotizar(peso, destino)	Calcular el costo estimado de un envío según su peso y destino.

generarDocumento()	Generar un documento relacionado con el envío, como una guía de transporte.
registrarAuditoria(evento)	Registrar las operaciones realizadas por el servicio logístico.

Clase: CertificadorFEL

Método	**Descripción**
consultarEstado()	Consultar el estado de una operación de certificación.
generarDocumento()	Generar un documento electrónico correspondiente a una factura.
registrarAuditoria(evento)	Registrar los eventos relacionados con la generación y certificación de documentos.

Clase: IntegracionController

Método	**Descripción**
cargarIntegraciones()	Crear y registrar las integraciones iniciales del sistema.
listarIntegraciones()	Obtener y mostrar las integraciones registradas junto con su estado de conexión.
buscarPorId(id)	Localizar una integración mediante su identificador y permitir consultar su configuración.
ordenarPorLatencia()	Ordenar las integraciones de acuerdo con su latencia promedio.
procesarCobro(api, monto)	Verificar si una integración implementa Transaccional y, de ser así, ejecutar el cobro correspondiente.

procesarReintegro(api, monto)	Verificar si una integración permite realizar reintegros antes de ejecutar la operación.

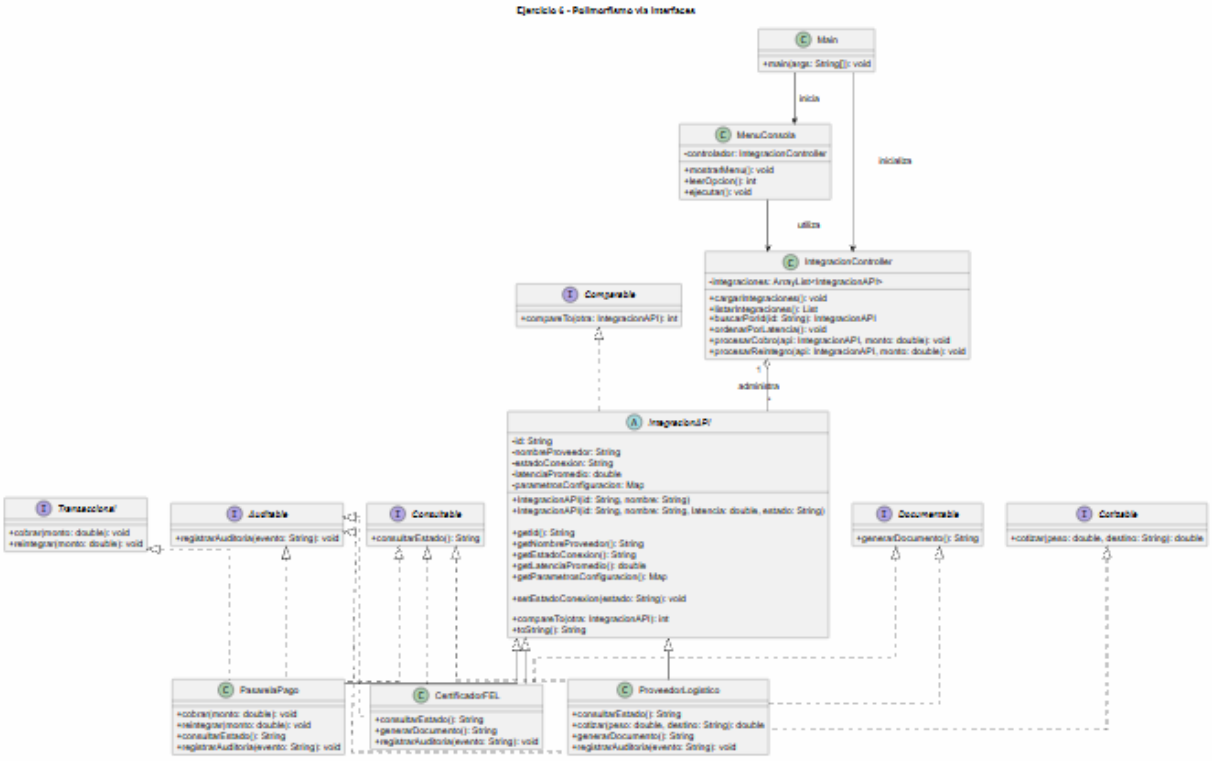
Clase: MenuConsola

Método	Descripción
mostrarMenu()	Presentar las opciones disponibles para administrar las integraciones.
leerOpcion()	Recibir y procesar la opción seleccionada por el administrador.
ejecutar()	Controlar el funcionamiento del menú y dirigir las solicitudes al controlador.

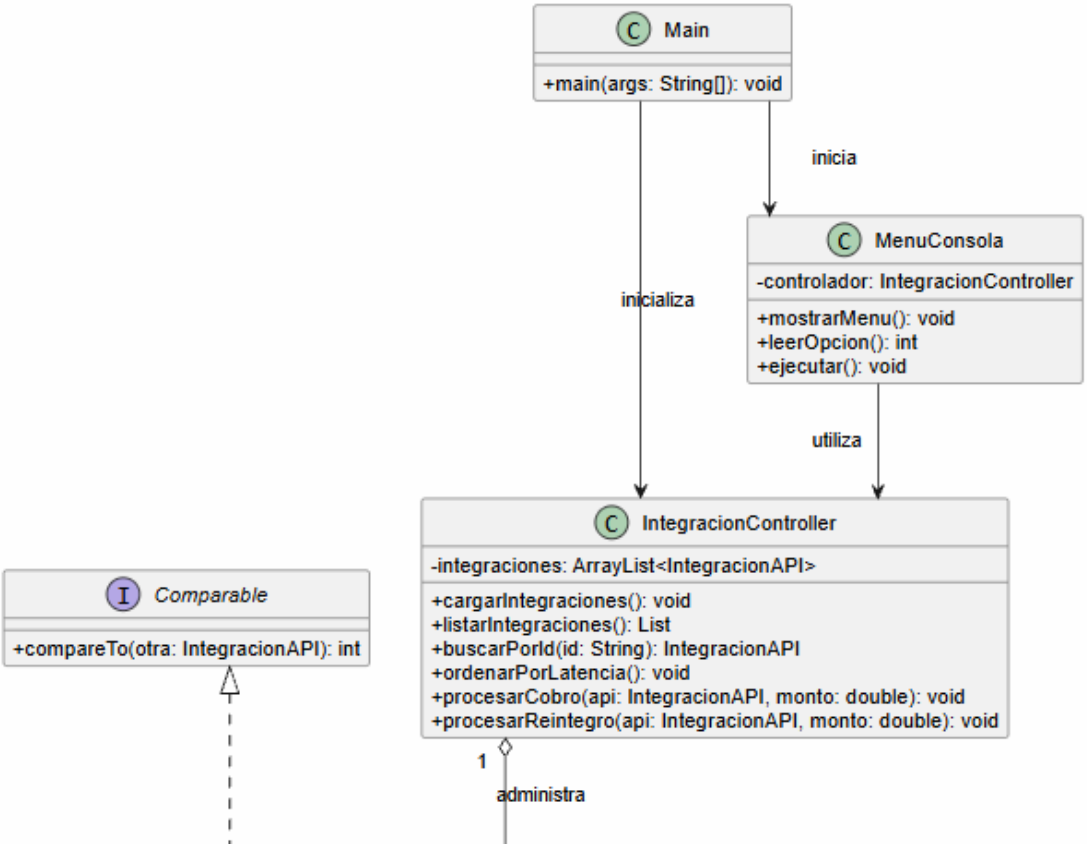
Clase: Main

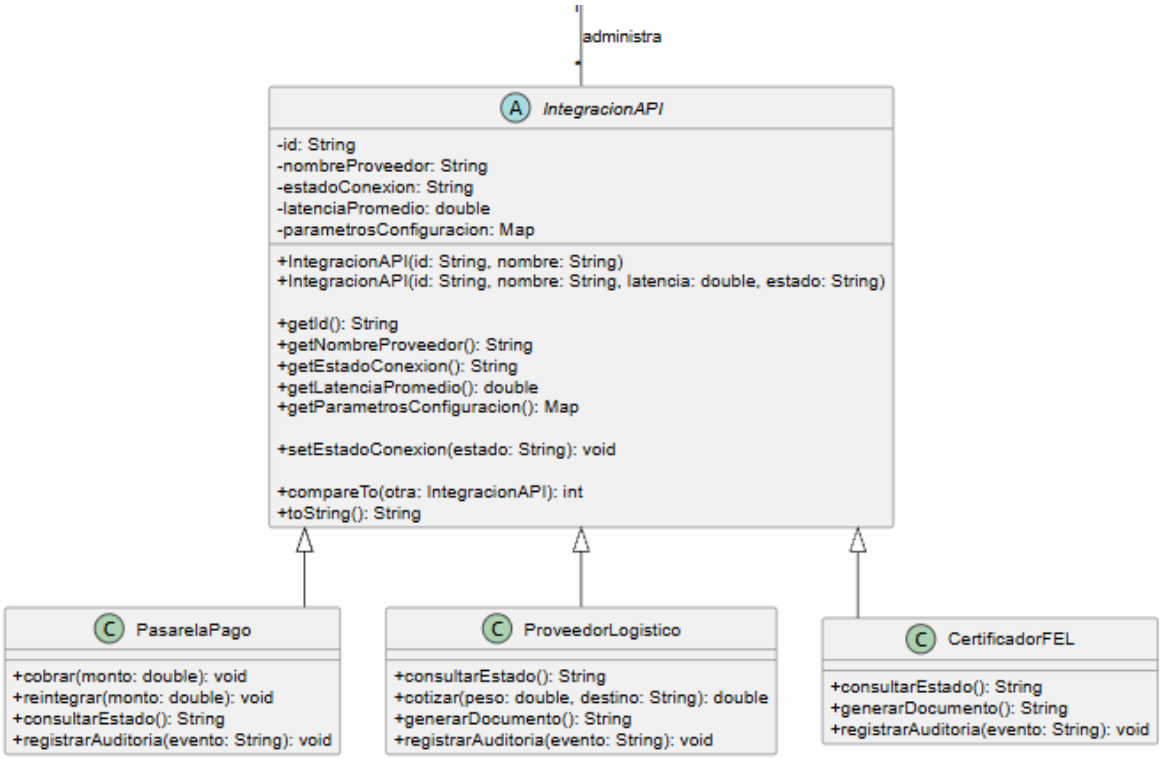
Método	Descripción
main(args)	Inicializar el controlador, realizar la carga inicial de integraciones, crear el menú e iniciar la ejecución del programa.

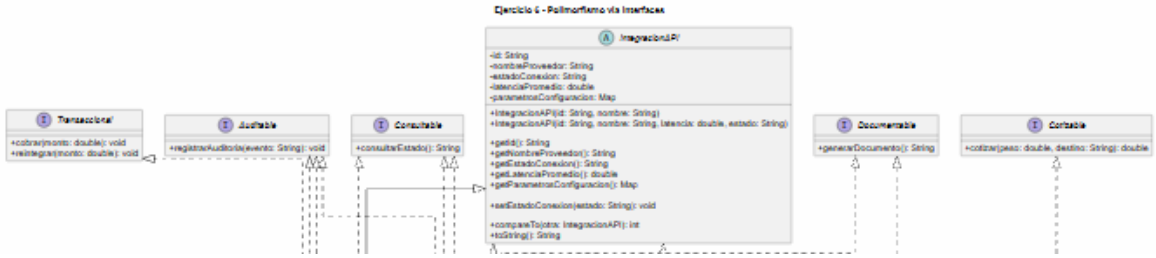
Diagrama UML



Ejercicio 6 - Polimorfismo via Interfaces




administra
IntegracionAPI
-id: String
-nombreProveedor: String
-estadoConexion: String
-latenciaPromedio: double
-parametrosConfiguracion: Map
+IntegracionAPI(id: String, nombre: String)
+IntegracionAPI(id: String, nombre: String, latencia: double, estado: String)
+getId(): String
+getNombreProveedor(): String
+getEstadoConexion(): String
+getLatenciaPromedio(): double
+getParametrosConfiguracion(): Map
+setEstadoConexion(estado: String): void
+compareTo(otra: IntegracionAPI): int
+toString(): String
PasarelaPago
+cobrar(monto: double): void
+reintegrar(monto: double): void
+consultarEstado(): String
+registrarAuditoria(evento: String): void
ProveedorLogistico
+consultarEstado(): String
+cotizar(peso: double, destino: String): double
+generarDocumento(): String
+registrarAuditoria(evento: String): void
CertificadorFEL
+consultarEstado(): String
+generarDocumento(): String
+registrarAuditoria(evento: String): void


Ejercicio 6 - Polimorfismo via Interfaces
IntegracionAPI
Transaccional
Auditable
Consultable
Documentable
Cotizable